

본립도생 (本立道生)



"기본이 바로 서면 나아갈 길이 생긴다"

어떤 일을 하든 기본과 원칙에 충실하면 발전
할 수 있으며 흔들리지 않는 근본이 곧 해법



황주현



2000.08.25.Busan



+82 010-4039-9111



hyunhwang091@naver.com



<https://github.com/Hwang-Ju-Hyun>

SKILLS



EDUCATION

2026.02	국립부경대학교 / 컴퓨터공학 졸업예정
2025.02	Digipen 수료 / (부산정보산업진흥원)

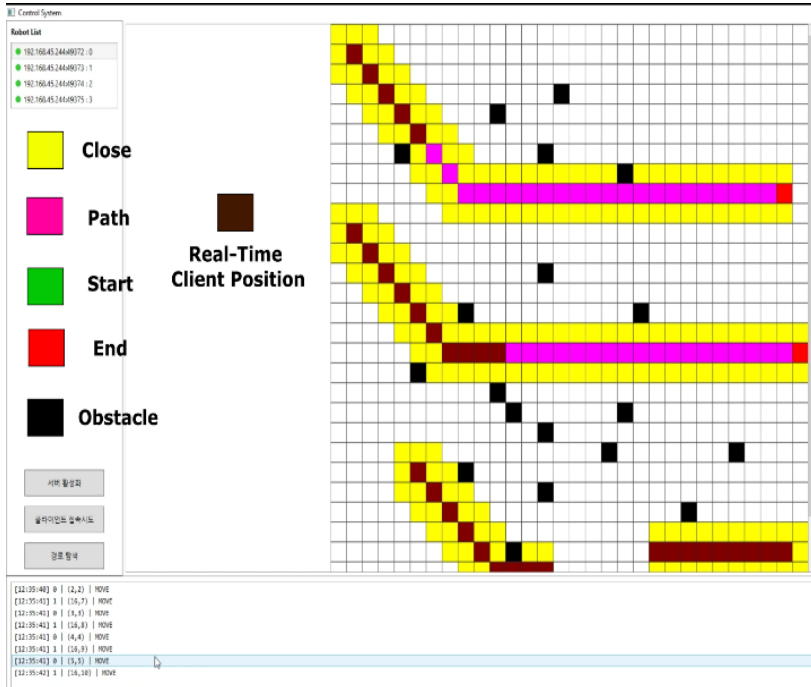
간단소개

- 단순히 기능을 구현하는 것을 넘어 시스템이 왜 이렇게 동작하는지 그 원리를 끝까지 파고듭니다. 해당 원리를 바탕으로 단기적인 해결책에 그치지 않고 시스템의 구조적 안정성과 추후 확장성까지 고려한 최적의 아키텍처를 직접 설계하고 검증하는 과정을 지향합니다.
- 직면한 문제를 해결하는 과정과 그 속에서 얻은 통찰을 기록하며 지식의 깊이를 더하려 노력합니다. 단순히 코드를 짜는 행위를 넘어 전체적인 시스템의 흐름을 스스로 설계하고 검증하며 기술에 대한 시야를 넓히는 데 집중합니다.
- 동료의 피드백을 기술적 성장의 기회로 삼는 높은 수용도를 갖추고 있습니다. 나의 논리를 고집하기보다 팀의 최선의 결과를 위해 유연하게 사고하며 전달함으로써 기술적 합의를 이끌어내는 과정에서 보람을 느끼고 추구합니다.

핵심강점

- **Concurrent Systems**: 비동기 및 멀티스레딩 TCP 통신 시스템을 직접 설계하여 안정적인 다중 클라이언트 환경 구축가능.
- **Network & Protocol**: TCP 스트림 특성을 고려한 커스텀 프로토콜 설계 및 패킷 파싱 로직 구현 경험.
- **Graphics & Engine**: C++과 OpenGL을 사용하여 추상화된 라이브러리 없이 그래픽스 파이프라인 직접 제어 및 하드웨어 자원 최적화를 통해 자체 게임 엔진 설계 경험.
- **System Architecture**: 메모리 관리 및 포인터에 대한 깊은 이해를 바탕으로 무결성 있는 객체지향 시스템 설계 가능.

프로젝트 경험 – Robot Monitor System



Robot Monitor System

실시간 로봇 관제 시스템

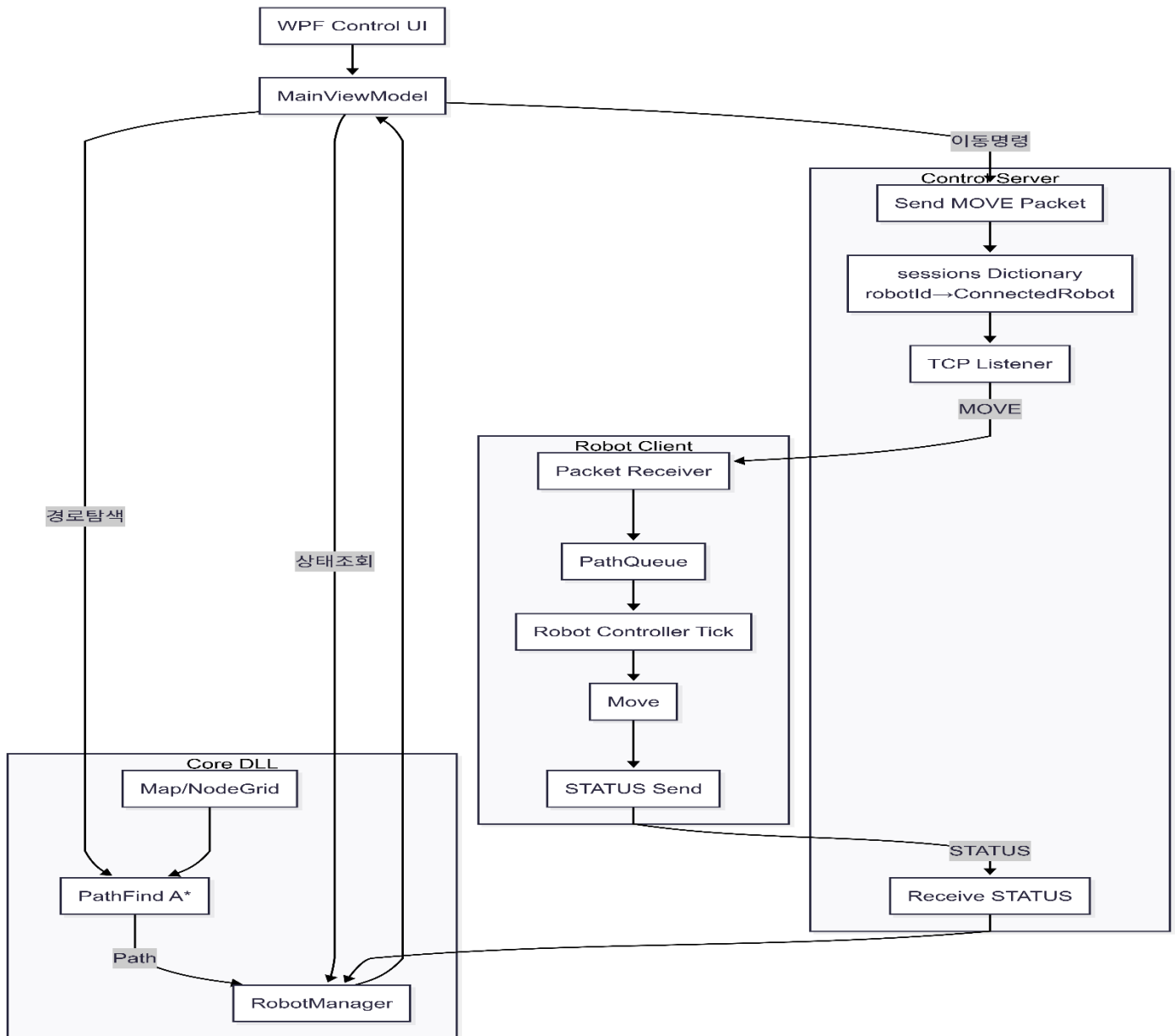
1. 프로젝트 개요

 진행 기간	2026.01 ~ 2026.01
 팀 규모	1 명
 역할	Programmer
 소속 기관	개인 프로젝트
 사용기술	C#(.Net) / WPF(UI) / Asynchronous / TCP/IP
 깃허브	https://github.com/Hwang-Ju-Hyun/RobotControlSystem
 최종결과물	https://youtu.be/teyKZmRD5fo

2. 프로젝트 소개

- 다중 이동 로봇(Client)을 서버 기반으로 관제하고 경로 탐색 및 상태를 실시간으로 모니터링이나 컨트롤 할 수 있는 시스템 입니다.
- 본 프로젝트는 다수의 로봇 클라이언트가 서버에 접속하여 자신의 상태(IDLE, MOVING, END)와 위치 정보를 주기적으로 전송하고 서버는 이를 수신하여 로봇별 상태 관리, 경로 명령 전송, 실시간 맵 시각화 및 로그 모니터링을 수행하는 관제 시스템입니다.
- A* 기반 경로 탐색 알고리즘을 직접 구현하였으며 비동기 TCP/IP 통신을 커스텀 프로토콜을 통해 실제 분산 환경을 가정한 구조로 설계하였습니다.

3. 프로젝트 구조 및 주요 구현 내용



1. 사용자 화면 (WPF 제어 센터)

- **데이터 중심 설계:** 화면과 내부 로직을 분리하여, 로봇이 늘어나도 화면이 끊기지 않고 실시간으로 상태를 보여주도록 설계했습니다.
- **실시간 맵 상호작용:** 마우스 클릭으로 각 로봇의 출발지와 목적지를 자유롭게 지정하고, 로봇의 이동 경로를 맵 위에 즉시 시각화했습니다.

프로젝트 경험 – Robot Monitor System

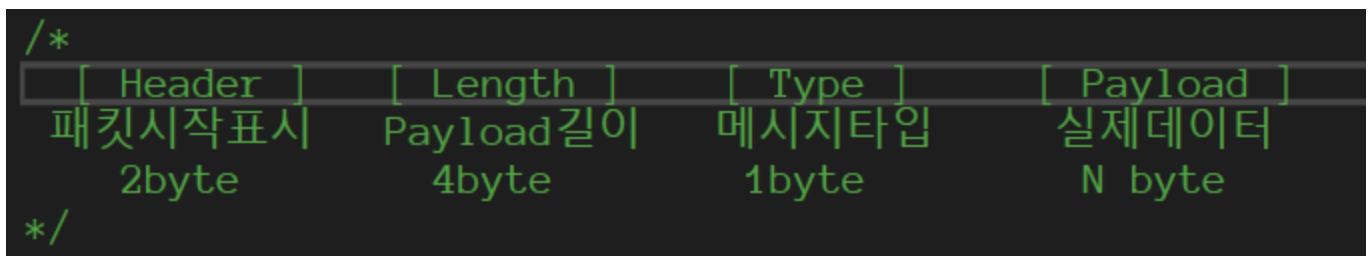
2. 시스템 핵심부 (공통 라이브러리 DLL) → [\[LINK\]](#)

- **통합 로봇 관리자:** 전역 관리 객체(Singleton)를 통해 접속된 모든 로봇의 ID, 위치, 상태를 한곳에서 관리하고 업데이트합니다.
- **경로 탐색 엔진:** A* 알고리즘을 직접 구현하여, 장애물을 피해서 목적지까지 가는 최단 경로를 실시간으로 계산합니다.
- **공통 모델 정의:** 로봇, 노드, 맵 데이터를 서버와 클라이언트가 똑같이 이해하도록 규격화하여 개발 효율을 높였습니다.

3. 서버 통신부 (비동기 TCP 서버) → [\[LINK\]](#)

- **커스텀 프로토콜 설계:** 데이터 누락을 막기 위해 헤더 | 길이 | 타입 | 데이터로 구성된 독자적인 통신 규격을 설계하고 적용했습니다.
- **다중 클라이언트 관리:** 여러 대의 로봇이 동시에 접속해도 서버가 멈추지 않도록 비동기 방식 (async/await)으로 메시지를 처리합니다.
- **데이터 직렬화:** 복잡한 로봇 정보를 바이트 단위로 쪼개고 합치는 과정을 직접 구현하여 통신의 신뢰성을 높였습니다.

↓ (커스텀 프로토콜 통신 규격)



4. 로봇 클라이언트 (가상 로봇 제어기)

- **명령 실행 제어기:** 서버에서 받은 경로 데이터를 순차적으로 처리하며 로봇의 실제 이동 상태를 서버에 보고합니다.
- **상태 보고 루프:** 주기적으로 현재 좌표와 이동 상태를 서버로 전송하여, 관제 화면과 실제 로봇의 위치를 실시간으로 동기화합니다.

4. 트러블 슈팅

서버/클라이언트 간 객체 역직렬화 실패 문제 🔍

■ 문제상황 🐛

서버와 클라이언트가 동일한 데이터 구조를 사용하도록 설계했음에도 네트워크로 전송된 객체를 복원하는 과정에서 타입 변환이 실패하는 문제가 발생했습니다.

디버그 로그상으로는 양쪽 모두 같은 라이브러리 버전(1.0.0)을 사용하고 있었기 때문에 원인을 쉽게 파악하기 어려웠습니다.

■ 원인분석 🕵️

문제의 핵심은 **버전 번호**가 아니라 **실제 빌드된 파일**이었습니다.

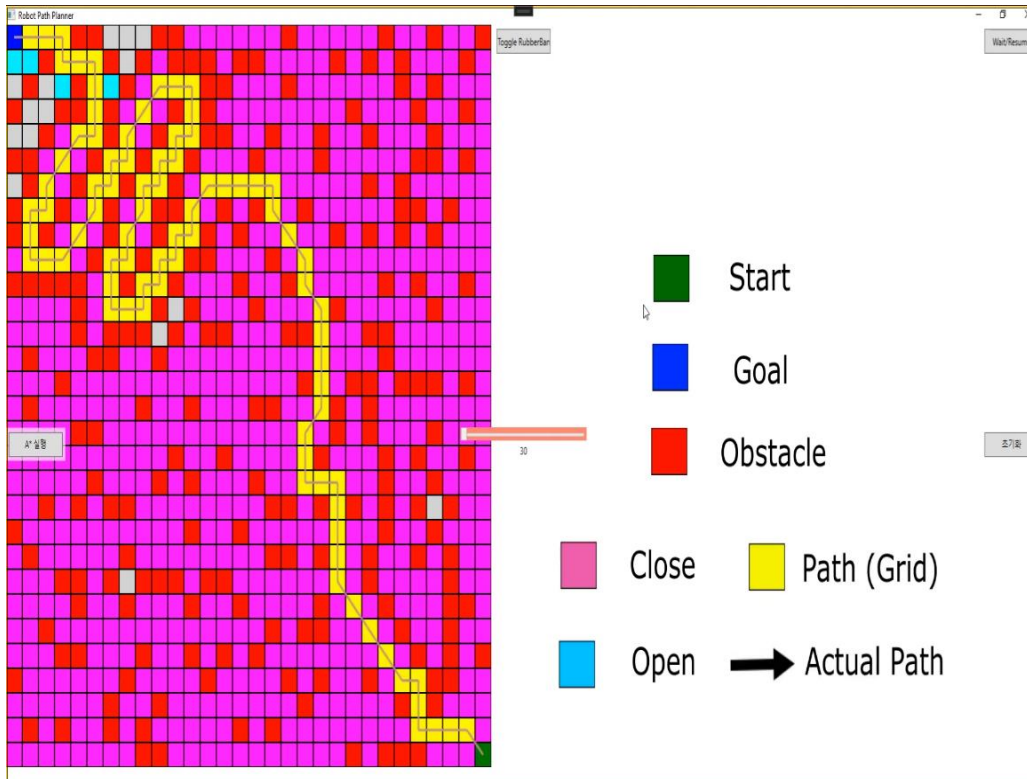
서버와 클라이언트가 같은 이름의 라이브러리, 같은 버전의 번호를 사용하고 있지만 서로 다른 시점에 각각 빌드된 DLL 파일을 참조하고 있었고 내부 식별 정보가 달라 런타임에서 **서로 다른 타입으로 인식**되었습니다. 즉, 겉보기에는 같은 구조체였지만 실행 환경에서는 “다른 종류의 객체”로 판단되어 변환에 실패했습니다.

■ 해결 방법 🛠️

- 공통 라이브러리를 **하나의 빌드 산출물로 통합**.
- 서버/클라이언트가 동일 DLL을 공유하도록 참조 방식 변경.
- 프로젝트 참조 방식으로 전환하여 빌드 결과가 항상 일치하도록 구성.

■ 결과 ⚡

- 타입 캐스팅 실패 재발 없음.
- 서버-클라이언트 간 데이터 변환 오류 완전 해결.



Path Find Simulator

경로 계획 시뮬레이터

1. 프로젝트 개요

진행 기간	2026.01 ~ 2026.01
팀 규모	1 명
역할	Programmer
소속 기관	개인 프로젝트
사용기술	C#(.Net) / WPF(UI) / Asynchronous
깃허브	https://github.com/Hwang-Ju-Hyun/PathFinder_Atar
최종결과물	https://youtu.be/tyGwd0AZXoo

2. 프로젝트 소개

- **격자(Grid)** 환경에서 장애물을 회피하여 최단 경로를 탐색하고 시각적 최적화를 수행하는 WPF 기반 **경로 계획 시뮬레이터**입니다.
- 단순히 최단 거리를 찾는 A* 알고리즘에 그치지 않고 실제 현장에서 객체가 주행할 때 발생하는 지그재그 이동의 비효율성을 해결하기 위해 **Rubber Banding** 기술을 도입하여 매끄러운 직선 주행 경로를 도출합니다.
- **실시간 지형 인지** : 격자 상에 실시간으로 배치되는 장애물을 감지하여 이동 가능 여부를 판단합니다.

3. 프로젝트 구조 및 주요 구현 내용

1. 실제 주행 환경을 고려한 단계적 경로 최적화 엔진 ($A^* \rightarrow \text{Theta}^*$) → [\[LINK\]](#)

- **Challenge:** 전통적인 A^* 알고리즘의 격자 기반 이동 제약으로 인해 로봇 주행 시 불필요한 꺾임 발생 및 물리적 주행 효율 저하
- **Solution:** 시야 검사(Line-of-Sight)를 적용한 *Theta 알고리즘*을 도입하여 격자 제약을 탈피하고, 직선 위주의 자연스러운 경로 도출
- **Result:** 알고리즘적 경로를 넘어 실제 구동 환경에 적합한 '물리적 최단 경로'를 산출했으며, 대각선 이동 시의 **Corner Cutting 예외 처리**로 시스템 신뢰성 확보

2. 경로 품질 검증 시뮬레이션 및 후처리(RubberBanding) 구축 → [\[LINK\]](#)

- **Challenge:** 격자 기반 경로 탐색은 알고리즘상 최단 경로를 제공하지만 단순한 탐색 결과만으로는 실제 로봇 주행에 적합한 경로인지 판단하기 어려움. 특히 장애물 근접 잦은 방향 전환 등은 시뮬레이션 없이 문제를 인지하기 쉽지 않음.
- **Solution:** 경로 탐색 과정을 Single-step 단위로 관찰할 수 있는 시각화 시뮬레이터를 구현하고 탐색 완료 후 RubberBanding 후처리를 적용하여 후처리 전·후 경로를 동일 조건에서 비교할 수 있도록 설계.
- **Technical Detail:** Manhattan, Euclidean 등 다양한 거리 계산 방식을 모듈화하여 휴리스틱 변화가 탐색 양상과 최종 경로 형태에 어떤 영향을 미치는지 단계별로 관찰·분석할 수 있는 구조를 구성.

3. 응답성 중심의 비동기 실시간 관제 시스템 (WPF)

- **Challenge:** 대규모 맵의 경로 연산 시 메인 스레드 점유로 인해 UI 프리징 현상이 발생하여 실시간 로봇 상태 추적 불가
- **Solution:** async/await 기반의 비동기 루프 아키텍처를 구축하여 연산과 UI 렌더링을 완전히 분리
- **Result:** 연산 중에도 실시간 명령 하달이 가능한 가용성을 확보했으며, 마우스 클릭 기반의 인터랙티브 인터페이스를 통해 **운영 편의성 극대화**

4. 트러블 슈팅

Grid 기반 A* 경로가 실제 이동에 부적합했던 문제

■ 문제상황

A* 알고리즘으로 생성된 경로가 이론적으로는 최단 경로였지만 다음과 같은 문제가 발생했습니다.

- 경로가 Grid 중심을 따라 이동.
- 불필요한 꺾임이 많음.
- 장애물을 거의 스치듯 통과하는 경로가 생성됨.

특히 시각화 결과 실제 로봇이 이동하기에는 **부자연스럽고 위험한 경로**라는 문제가 드러났습니다.

■ 원인분석

Grid 기반 A*는 **이산 공간(discrete space)** 을 전제로 설계되어있었고 실제 로봇 이동은 **연속 공간(continuous space)**에서 이루어집니다. 하지만 8 방향 이동 + Grid 비용 계산은 “이동 가능 여부”는 판단하지만 “이동 품질”까지는 보장하지 못합니다.

즉 알고리즘은 맞지만 모델링이 현실과 어긋나 있음을 확인했습니다.

■ 해결 방법

기존 A* 구조를 유지하면서 연속 공간 이동을 고려할 수 있는 Theta* 개념을 도입하였습니다.

- 휴리스틱을 Euclidean Distance로 변경.
- 노드 확장 시 부모 노드의 부모와 다음 노드 간 Line-of-Sight 검사 수행.
- 직선 이동이 가능하면 중간 노드를 생략하도록 구현.

■ 결과 ⚡

- 경로의 꺾임이 크게 감소.
- 실제 로봇 이동에 가까운 자연스러운 경로 생성.
- Grid 기반 구조를 유지하면서도 연속 공간 효과 확보

Rubber Banding 적용 시 충돌을 감지하지 못하던 문제 🔍

■ 문제상황 🐛

Rubber Banding을 적용해 경로를 단순화하던 중 장애물을 가로지르는 경로임에도 불구하고 충돌이 없다고 판단되는 오류가 발생하였습니다.

■ 원인분석 🕵️

Line-of-Sight 검사에서 Bresenham 알고리즘을 단순히 셀 중심 기준으로만 검사하였었기 때문에 대각선 이동 시 실제로는 장애물 모서리를 통과하지만 알고리즘상으로는 충돌하지 않는 경우 발생하였습니다. 즉 **Grid 셀 단위 충돌 검사 한계**가 원인이었습니다.

■ 해결 방법 🚧

Bresenham 알고리즘 모듈이 진행 되는 중에 대각선 이동이 발생하는 경우 인접한 두 셀(가로 / 세로)을 추가로 검사하고 코너 컷(corner cutting)을 명시적으로 차단하도록 보완하여 해결하였습니다.

■ 결과 ⚡

장애물을 스치거나 관통하는 경로 제거되면서 Rubber Banding 이후에도 안정적인 충돌 회피 보장할 수 있게 되었습니다.



Wothingthing2

자체 엔진을 활용하여 구현한 게임

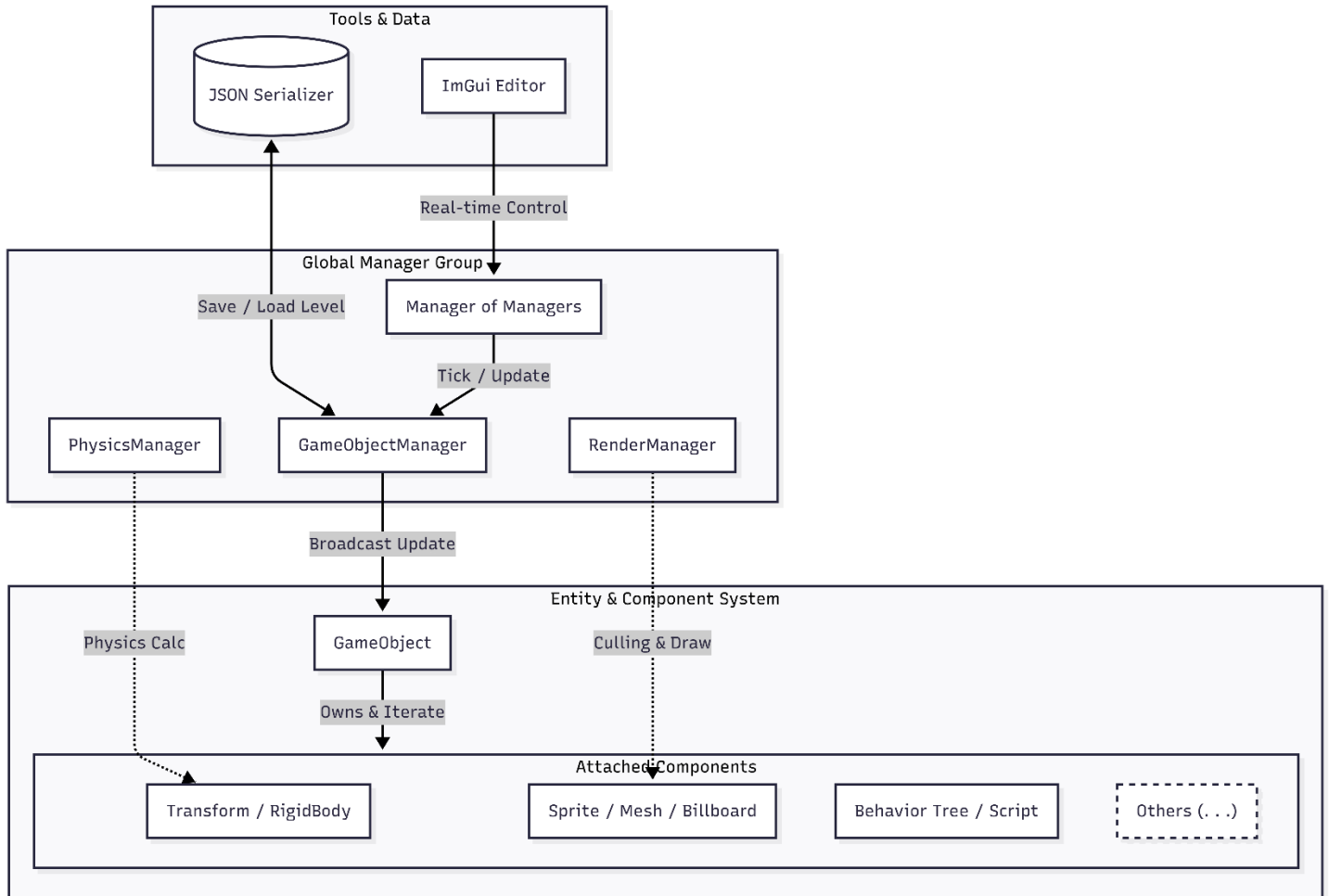
1. 프로젝트 개요

 진행 기간	2025.04 ~ 2025.12
 팀 규모	1 명
 역할	Programmer
 소속 기관	국립 부경대학교
 사용기술	C++ / OpenGL(GLFW) / ImGui / ImGuiZMO / ASSIMP / JSON / Visual Leak Detector
 깃허브	https://github.com/Hwang-Ju-Hyun/BadaBoomkyong
 최종결과물	https://youtu.be/aEGslSrzbml

2. 프로젝트 소개

- Wothingthing2는 솔로 프로젝트로 개발한 2D 횡스크롤 + 3D 배경 게임입니다.
- 배경은 3D로 구성하고 플레이어와 몬스터는 2D 스프라이트로 구현하여 3D/2D 혼합 환경에서의 게임 경험을 제공합니다.
- 본 프로젝트를 통해 처음으로 OpenGL 기반 직접 렌더링을 경험했고, 이 과정을 통해 렌더링 파이프라인, 자료 구조 설계, 패턴 적용 등 엔진 없이 게임을 개발하는 경험과 전문성을 쌓을 수 있었습니다.

3. 프로젝트 구조



1. 컴포넌트 기반 엔티티 구조 (Component-Based Entity System)

- **유니티 방식의 컴포넌트 기반 구조** : 모든 객체는 GameObject라는 컨테이너에 독립적인 기능을 가진 컴포넌트들을 부착(Attach)하여 구성됩니다.
- **동적 업데이트 전파 (Broadcasting Update)**: GameObjectManager로부터 Tick 신호를 받은 각 객체는 자신에게 포함된 Transform, Sprite, Rigidbody, BT AI 등 수십 개의 컴포넌트 리스트를 순회하며 Update() 로직을 실행합니다.
- **높은 확장성과 유연성**: 객체의 클래스를 매번 새로 만들지 않고 이미 검증된 컴포넌트들의 조합만으로 새로운 유닛이나 오브젝트를 즉시 생성할 수 있는 구조를 구축했습니다.

2. OpenGL 기반 커스텀 렌더링 파이프라인 (Rendering Pipeline)

- **데이터 수집 및 정렬** : RenderManager가 매 프레임 모든 객체의 Mesh와 Material 정보를 수집하며, 특히 투명도(Alpha)를 가진 오브젝트들을 카메라 거리순으로 정렬하여 렌더링 순서를 제어합니다.
- **셰이더 및 버퍼 직접 제어** : 상용 엔진의 블랙박스를 배제하고, Vertex Array Object(VAO), Vertex Buffer Object(VBO) 생성부터 GLSL 셰이더 로딩, Uniform 변수 전달까지 그래픽스 파이프라인 전 과정을 직접 통제합니다.
- **3D 환경 가시화와 행렬 공간 변환** : 모델의 지역 좌표를 월드에 배치(Model)하고, 카메라를 기준으로 관찰자 시점을 확립(View)한 뒤 원근 투영(Projection)을 통해 3차원 데이터를 클립 공간의 2차원 좌표로 수렴시켜 수학적인 원근감과 깊이감을 가시화했습니다.

3. 실시간 편집 및 데이터 파이프라인 (Editor & Data)

- **ImGui 기반 실시간 튜닝** : 엔진 내부에 ImGui를 통합하여 런타임 중에도 컴포넌트의 수치(위치, 속도, AI 상태 등)를 즉시 수정하고 결과를 확인하는 개발 도구를 구축했습니다.
- **JSON 기반 데이터 영속성** : 에디터에서 편집된 레벨 및 객체 정보를 JSON Serializer를 통해 직렬화하여 저장하고 불러옴으로써, 데이터 기반(Data-driven)의 게임 월드 구성이 가능하도록 설계했습니다.

4. 주요 구현 내용

1. 비동기 인터럽트가 가능한 Behavior Tree 프레임워크 설계 → [\[LINK\]](#)

- **Challenge**: 특정 몬스터의 행동 패턴이 고도화됨에 따라 기존 FSM(Finite State Machine) 구조로는 상태 전이가 기하급수적으로 복잡해지는 '상태 폭발' 문제와 유지보수의 한계 직면
- **Solution**: 계층적 구조로 복잡한 로직을 모듈화할 수 있는 **범용 BT 프레임워크**를 직접 설계하고 실시간 Abort 기능을 추가하여 비동기적 인터럽트 구조 구현
- **Result**: 10종 이상의 복잡한 보스 패턴을 조건부 노드 결합만으로 설계 가능하게 하여 **코드 복잡도를 획기적으로 낮추고**, 플레이어의 반응에 즉각 대응하는 유연한 AI 환경 구축

2. Blackboard & 메시지 시스템을 통한 데이터 공유 최적화 → [\[LINK\]](#)

- **Challenge** : AI 노드 간 빈번한 데이터 참조로 인한 포인터 탐색 비용 증가 및 시스템 결합도 상승 문제 인식

- **Solution**: 모든 노드가 접근 가능한 공유 메모리(Blackboard)와 비동기 메시지 큐 시스템 구현
- **Technical Detail**: Blackboard 내부 참조 캐싱을 적용해 매 프레임 발생하는 불필요한 연산을 제거하고, 메시지 기반의 이벤트 처리로 AI 로직 간 독립성 확보

3. 수학적 모델링 기반의 Raycast 및 충돌 판정 시스템 → [\[LINK\]](#)

- **Challenge**: 물리 엔진 라이브러리 없이 정교한 오브젝트 선택 및 투사체 충돌 판정 시스템 구현 필요
- **Solution**: 광선의 매개변수 방정식과 Slab 교차 방식을 활용한 Ray-AABB 충돌 알고리즘 직접 구현
- **Result**: 정규화 및 평행 레이 등 예외 케이스를 처리하는 안정적인 Closest-hit 판정 시스템을 완성하여 물리적 상호작용의 신뢰성 확보

4. 3D 환경 몰입감 향상을 위한 커스텀 렌더링 기법 → [\[LINK\]](#)

- **Challenge**: 3D 공간 내 2D 스프라이트의 왜곡 현상 방지 및 자연스러운 배경 원근감 구현
- **Solution**: 카메라 방향을 상시 추적하는 빌보드(Billboard) 렌더링과 동적 시점 적용이 가능한 Skybox 직접 구현
- **Technical Detail**: 렌더링 순서(Rendering Order)를 고려한 파이프라인 설계를 통해 불투명/투명 오브젝트 간의 깊이 정합성을 유지하며 게임의 시각적 몰입도 향상.

5. 트러블 슈팅

투명/불투명 오브젝트 렌더링 문제

■ 문제상황

개발 초기에 투명 오브젝트가 불투명 오브젝트를 가리는 현상이 발생했습니다.

■ 원인분석

알파 블렌딩 적용 시 투명 오브젝트가 뒤의 불투명 오브젝트를 가리거나 사라지는 깊이 판정(Depth Testing) 오류가 발생하였습니다. 해당 원인 GPU의 기본 Depth Test 로직은 가장 가까운 픽셀만

기록하므로 투명 객체가 먼저 그려질 경우 뒤쪽의 유효한 픽셀 정보가 폐기되는 파이프라인의 구조적 한계인 것을 인지하였습니다.

■ 해결 방법 🚧

RenderManager 내부에 단계적 렌더링 패스(Multi-pass Rendering) 구조 도입하였습니다.

- **Opaque**: 모든 불투명 오브젝트를 Depth Test 활성화 상태로 우선 렌더링하여 기본 배경 정보 확립.
- **Transparent**: 투명 오브젝트들을 카메라와의 거리에 따라 **뒤에서 앞으로(Back-to-Front)** 정렬 후 최종 합성 수행.

■ 결과 ⚡

- 투명 오브젝트가 불투명 오브젝트를 가리는 문제가 해결되었습니다.
- 자연스러운 반투명 효과와 씬 렌더링 안정성 확보하였습니다.

AI 구현과 상태 관리 문제 🔍

■ 문제상황 🐛

FSM(Finite State Machine) 기반 AI 구현 중, 상태 클래스(Idle, Ranged 등) 내부에 몬스터별 공격 방식에 따른 조건문(if/else)이 기하급수적으로 증가하며 유지보수 효율 저하되었습니다.

■ 원인분석 🕵️

상태 전환 로직과 구체적인 행동 로직이 한 곳에 섞여 발생하는 **단일 책임 원칙(SRP)** 위반 및 신규 몬스터 추가 시 기존 코드를 대폭 수정해야 하는 **OCP(개방-폐쇄 원칙)** 위반 확인되었습니다.

■ 해결방법 🚧

- **행동 추상화**: IRangedAttackBehavior 등 공통 인터페이스를 정의하여 행동 로직을 독립적인 전략 객체로 캡슐화하였습니다.
- **위임 구조 설계**: 상태 클래스는 전환 흐름만 제어하고, 실제 행동은 런타임에 주입된 전략 객체에 위임하도록 재설계하였습니다.

■ 결과 ⚡

- 생산성 향상: 신규 몬스터 추가 시 수정 파일 수 **75% 감소하였습니다.** (4개 → 1개)
- 유지보수 효율: AI 관련 코드 수정량 약 **40% 절감** 및 조건문 제거를 통한 로직 가독성 2배 향상하였습니다.

더욱 많은 개인 및 협업 프로젝트와 공부 내용이 있습니다!

[\[LINK\]](#) 

감사합니다.